Kubernetes

Persistent Volumes

Storage Requirements

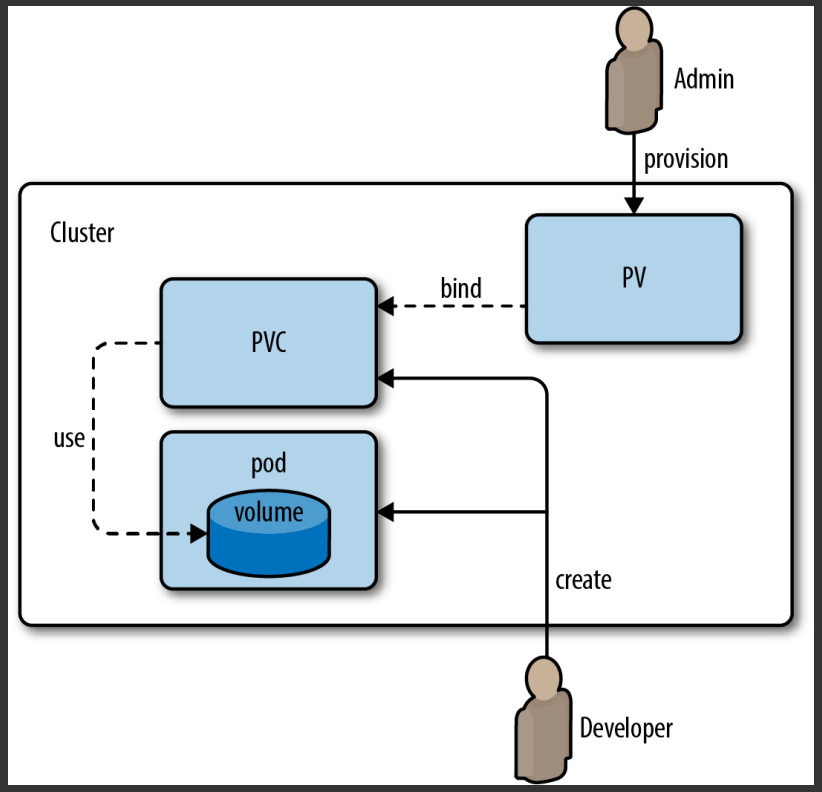
1. Storage that doesn’t depend on the pod lifecycle
2. Storage must be available on all nodes
3. Storage needs to survive even if cluster crashes

For these requirements, we need Persistent volumes.

Volumes vs Persistent Volumes

Volumes exist in the context of specific Pods. A Volume's lifecycle is coupled to the Pod that owns it. It allows safe use of storage by all the containers in the Pod, but the volume's data will still be lost when the Pod terminates.

Persistent Volumes build upon the foundations established by Volumes. They provide storage that's decoupled from Pods, allowing data to persist beyond the lifecycle of individual Pods.



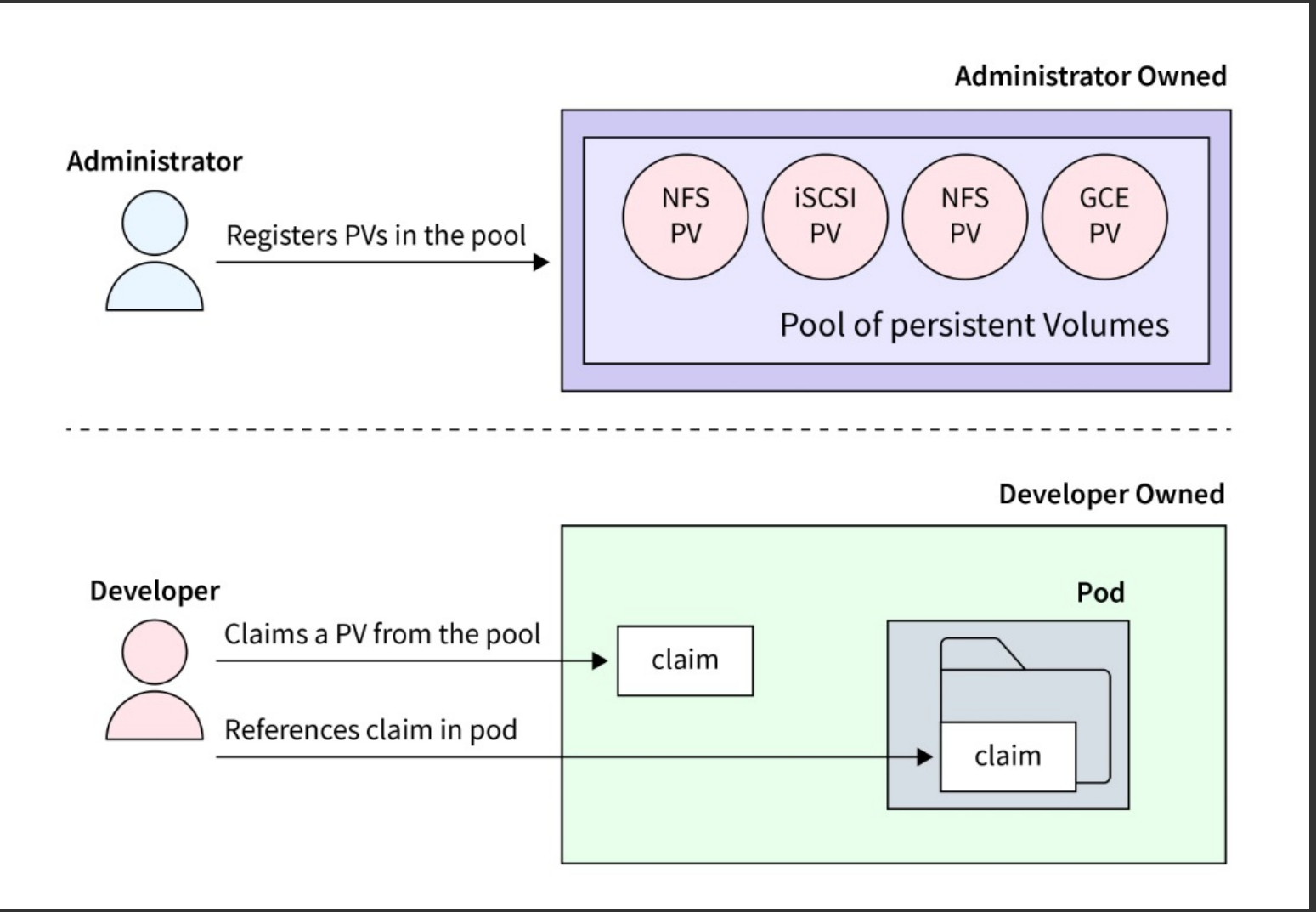
Types of Persistent Volumes

hostPath – This sort of Persistent Volume makes use of a directory on the host machine's filesystem. It is suitable for testing and development but not for production usage since it lacks data replication and dynamic provisioning.

nfs – Used to access Network File System (NFS) mounts. NFS allows files to be stored in a central location (a file server) and shared across different client systems over a network.

csi – Allows integration with storage providers that support the Container Storage Interface (CSI) specification, such as the block storage services provided by cloud platforms like Amazon Elastic Block Store (EBS) or Azure Disk.

PV and PVC


Administrator Owned
Administrator
Registers PVs in the pool
NFS PV
iSCSI PV
NFS PV
GCE PV
Pool of persistent Volumes
Developer Owned
Developer
Claims a PV from the pool
References claim in pod
claim
Pod
claim

The lifecycles of PVs and PVCs

1. **Provisioning**: the PV is created and its storage is allocated using the selected driver.

2. **Binding**: Kubernetes automatically watches for new PVCs and binds them to the PVs they reference. Each PV can only be bound to a single PVC at a time.

3. **Using**: A volume enters use once its PVC is consumed by a Pod. In this state, the PV is actively providing storage to an application in your cluster.

4. **Reclaiming**: Users can delete the PVC to relinquish access to the PV. When this happens, the storage used by the PV is “reclaimed.”

PV and PVC

```yaml
apiVersion: v1
kind: PersistentVolume
metadata:
  name: demo-pv
spec:
  accessModes:
    - ReadWriteOnce
  capacity:
    storage: 1Gi
  storageClassName: standard
  hostPath:
    path: /tmp/demo-pv
```

```yaml
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: demo-pvc
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 1Gi
  storageClassName: standard
  volumeName: demo-pv
```

Examples of PV Configs

```yaml
apiVersion: v1
kind: PersistentVolume
metadata:
  name: gcp-pd-pv
spec:
  capacity:
    storage: 50Gi
  accessModes:
    - ReadWriteOnce
  gcePersistentDisk:
    pdName: my-gcp-disk
    fsType: ext4
  persistentVolumeReclaimPolicy: Delete
```

Persistent Volume for
Google Persistent Disk

```yaml
apiVersion: v1
kind: PersistentVolume
metadata:
  name: csi-pv
spec:
  capacity:
    storage: 5Gi
  accessModes:
    - ReadWriteOnce
  csi:
    driver: ebs.csi.aws.com
    volumeHandle: vol-0123456789abcdef
    fsType: ext4
  persistentVolumeReclaimPolicy: Delete
```

Persistent Volume for CSI
(Container Storage Interface)

```yaml
apiVersion: v1
kind: PersistentVolume
metadata:
  name: hostpath-pv
spec:
  capacity:
    storage: 1Gi
  accessModes:
    - ReadWriteOnce
  hostPath:
    path: /data/hostpath  # Path on the
  persistentVolumeReclaimPolicy: Retain
```

Persistent Volume for HostPath

Access Modes

For PVs, Kubernetes provides three access modes: ReadWriteOnce, ReadOnlyMany, and ReadWriteMany.

1. ReadWriteOnce (RWO) - the volume can be mounted as read/write by a single node. Can be used for single instance or sharded DB storage.
2. ReadOnlyMany (ROX) - the volume can be mounted as read-only by many nodes. Can be used for Read replicas of DB.
3. ReadWriteMany (RWX) - the volume can be mounted as read/write by many nodes. Can be used for logging, data aggregation, NFS.

Dynamic provisioning with StorageClass

- A StorageClass allows you to specify different types of storage (e.g., SSDs, HDDs) and their configurations, like performance settings or zones.
- StorageClasses specify provisioners that manage the actual storage, typically tied to cloud provider-specific or on-premises storage solutions.

Here are some widely used provisioners:

AWS EBS (kubernetes.io/aws-ebs): Amazon Elastic Block Store volumes.

GCE PD (kubernetes.io/gce-pd): Google Compute Engine Persistent Disk.

Azure Disk (kubernetes.io/azure-disk): Microsoft Azure Disk storage.

NFS (nfs): Network File System for shared, read-write access.

Local (kubernetes.io/no-provisioner): Used for local storage volumes.

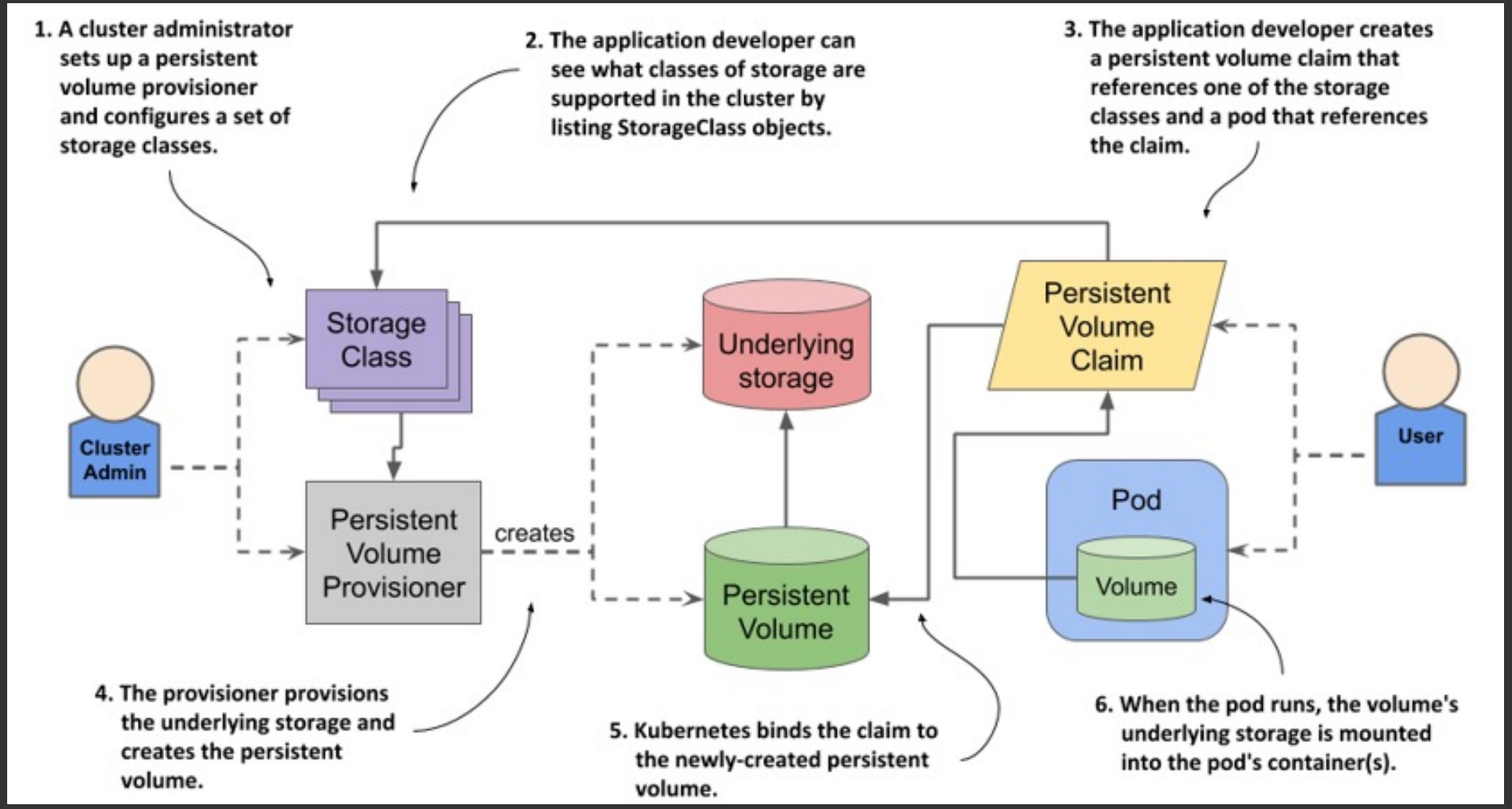
Dynamically provisioning PVs

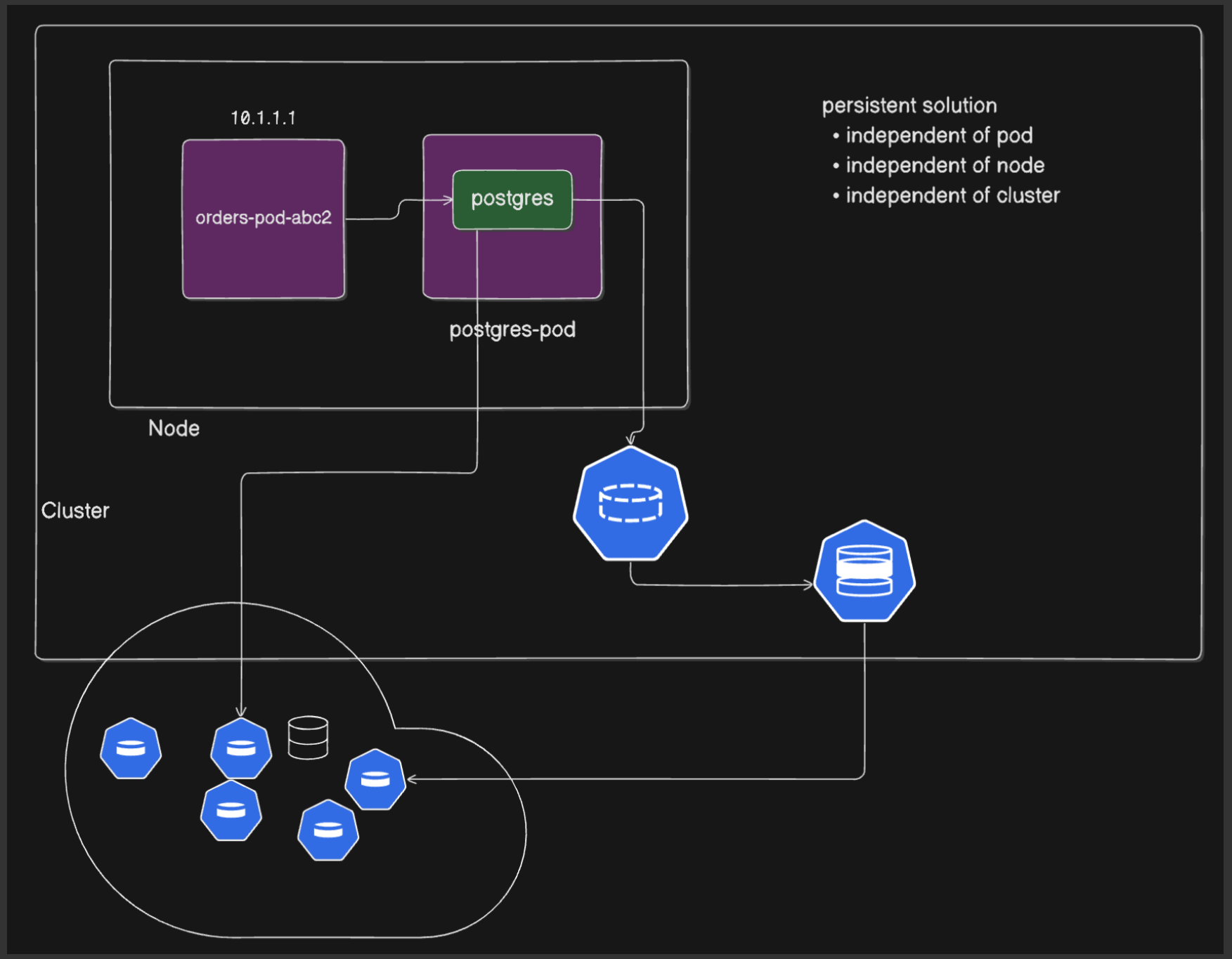
Because the volumeName is omitted from the PVC's spec, the PV is dynamically provisioned:

```yaml
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: pvc-dynamic
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 1Gi
  storageClassName: standard
```

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: pvc-pod
spec:
  containers:
    - name: pvc-pod-container
      image: nginx:latest
      volumeMounts:
        - mountPath: /data
          name: data
  volumes:
    - name: data
      persistentVolumeClaim:
        claimName: pvc-dynamic
```

Dynamic Provisioning




10.1.1.1
orders-pod-abc2
postgres
postgres-pod
Node
Cluster
persistent solution
• independent of pod
• independent of node
• independent of cluster